Background

Opening. This presentation translates our June 2026 INCOSE INSIGHT article into twenty practical recommendations for engineering work. The unifying idea is that generative AI should be treated as part of a disciplined engineering process, with explicit specifications, test cases, verification, and accountability. It is not merely a faster way to produce text.

Context. Engineers are already using AI to summarize documents, draft requirements, evaluate alternatives, and organize engineering work. The risks include missing context, silent assumptions, unsupported quantitative answers, weak traceability, and rework. We advocate a measurable process in which the human defines the problem and validates the results.

Dedication. The article honors Dr. Barclay Brown, who emphasized the practical use of AI while preserving the judgment and responsibility of systems engineers. The following examples are illustrative applications of the article's advice.

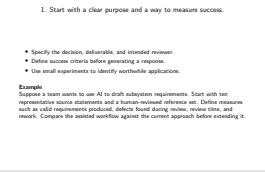
Background

This presentation is based on the following INCOSE INSIGHT special issue article with additions:

Ray Madachy, Dan O'Leary, Tom McDermott, and Sinan Bank. *Top Twenty Tips for Using Generative AI in Systems Engineering. INCOSE INSIGHT, June 2026.* https://incose.onlinelibrary.wiley.com/doi/10.1002/inst.70053



1. Start with a clear purpose and a way to measure success.


1. Start with a clear purpose and a way to measure success.

- Specify the decision, deliverable, and intended reviewer.
- Define success criteria before generating a response.
- Use small experiments to identify worthwhile applications.

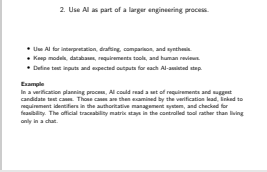
Example
Suppose a team wants to use AI to draft subsystem requirements. Start with ten representative source statements and a human-reviewed reference set. Define measures such as valid requirements produced, defects found during review, review time, and rework. Compare the assisted workflow against the current approach before extending it.

Begin with the engineering problem and a measure of success, not with the tool.
Explanation. This is a systems engineering problem-definition exercise. The question is not simply where a chatbot might be used, but which engineering decision or artifact can be improved, by how much, and under what constraints. Establish the baseline process, the expected output, and who will accept the result. An attractive demonstration is not evidence of value if it does not improve an engineering outcome.

Illustrative example. Assess whether AI-assisted requirement drafting reduces review rework.

Presenter emphasis. The measurable target is better engineering work, not simply faster text generation.

2. Use AI as part of a larger engineering process.

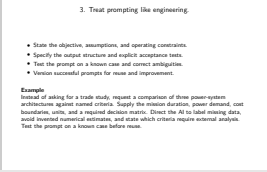


Integrate AI at defined steps of an existing workflow, as a component rather than the process itself.
Explanation. A language model is a component of an engineering process, not the entire process. It is especially useful where the work involves interpreting text, reorganizing information, and proposing alternatives. Activities that require controlled data, execution, authoritative records, or formal approval remain in the appropriate engineering tools and organizational workflow.

Illustrative example. Draft verification cases with AI, then manage and approve them in the test system.

Presenter emphasis. Show the audience where the AI step starts, where it ends, and who accepts the handoff.

3. Treat prompting like engineering.



A prompt is a small specification and should be testable.

Explanation. A vague request forces the model to supply its own assumptions, which may be inconsistent with project intent. Treat the prompt as an engineering input: identify the task, scope, definitions, constraints, expected artifact, and how that artifact will be evaluated. Preserve successful prompt versions so team members can reproduce and improve the method rather than restarting from an informal conversation.

Illustrative example. Specify units, assumptions, table fields, and quality checks in a trade-study prompt.

Presenter emphasis. A useful meta-prompt is: How could this instruction be made less ambiguous before execution?

4. Manage context deliberately across sessions and keep a record of AI-assisted work.

4. Manage context deliberately across sessions and keep a record of AI-assisted work.

- Record inputs, sources, assumptions, outputs, and decisions.
- Preserve accepted artifacts and rejected alternatives.
- Reestablish the approved baseline when work resumes.

Example
A team drafting an interface control document maintains a repository entry with the approved interface definition, the source documents, the prompt template version, the generated proposal, and reviewer comments. On the next session, the engineer attaches the current baseline and asks the model to identify only the differences. The process avoids rebuilding decisions from memory.

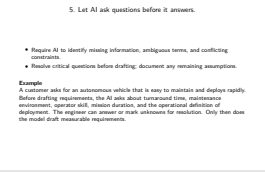
A chat history is not a controlled baseline; preserve context and provenance deliberately.

Explanation. AI interaction is easy to lose or misunderstand when a session ends, a model changes, or another engineer takes over. Preserve enough provenance to determine what information the model had, what it produced, which parts were accepted, and who decided. Durable project artifacts should contain the current assumptions, glossary, constraints, sources, and reusable prompts. Conversation history by itself is not a configuration-management system.

Illustrative example. Store prompt version, requirements baseline, model output, and reviewer disposition.

Presenter emphasis. This tip is about traceability and configuration control, not storing every chat message.

5. Let AI ask questions before it answers.



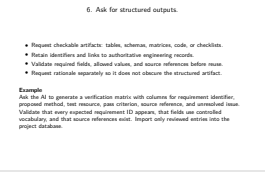
Make the model ask before it assumes.

Explanation. The model sees a finite body of context and may silently invent plausible assumptions when something essential is missing. Reverse the interaction: invite it to interview the engineer before it produces the deliverable. Clarifying scope, definitions, interfaces, units, and acceptance thresholds early can prevent an apparently complete but invalid result. Questions are especially valuable when the prompt contains terms with multiple interpretations.

Illustrative example. Have AI clarify what "rapid deployment" means before drafting requirements.

Presenter emphasis. A short prompt that often helps is: What must you know before proceeding?

6. Ask for structured outputs.



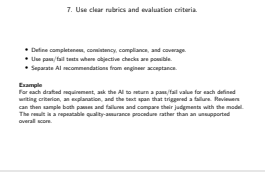
Choose formats that can be easily checked, compared, and reused.

Explanation. Engineering products often need to be inspected by another person or transferred into another tool. Free-form prose can hide omissions, inconsistent terminology, and broken links. A schema or table makes required fields explicit and enables basic automated checks. Narrative reasoning can still be useful, but it should supplement rather than obscure the artifact being produced.

Illustrative example. Return requirement ID, source, verification method, and review status as CSV.

Presenter emphasis. A structured response is easier to verify, but the structure alone does not make it correct.

7. Use clear rubrics and evaluation criteria.



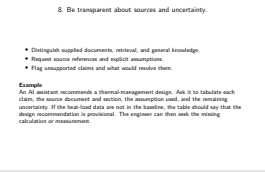
Replace "Is it good?" with explicit, reviewable tests.

Explanation. General judgments of quality are subjective and can change across model runs. A rubric converts quality into observable characteristics. For requirements, the criteria might include singularity, lack of ambiguity, feasibility, traceability, verification method, and consistency with the governing guidance. The model can help apply a rubric, but the rubric and the final acceptance decision should remain controlled by the engineering team.

Illustrative example. Evaluate requirements against the INCOSE Guide to Writing Requirements.

Presenter emphasis. Use the same criteria to compare alternative AI models and prompt revisions.

8. Be transparent about sources and uncertainty.



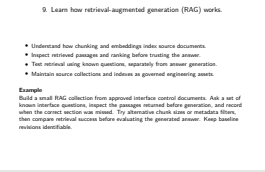
Every claim should arrive labeled as sourced, assumed, or unverified.

Explanation. An authoritative-sounding answer can combine facts from project materials, retrieved external sources, background model knowledge, and unstated assumptions. Reviewers need to know which is which. Require explicit source attribution where possible and have the model disclose uncertainty or absent evidence. A source citation must itself be checked; a citation-looking string is not proof that the source contains the claim.

Illustrative example. For each design claim, identify its source requirement or mark it “unverified.”

Presenter emphasis. Transparency supports calibrated trust; it does not remove the obligation to verify.

9. Learn how retrieval-augmented generation (RAG) works.



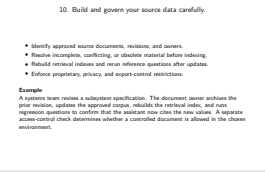
Know the retrieval pipeline well enough to test whether the right evidence reached the model.

Explanation. Retrieval-augmented generation supplies selected document passages to the language model at the time of a request. The overall response depends not only on model quality but also on document selection, segmentation into chunks, indexing, and ranking. If the relevant evidence is never retrieved, the generator cannot reliably use it. Engineers should learn enough about these mechanisms to test coverage, relevance, and source currency.

Illustrative example. Answer an interface question using retrieved sections of approved ICDs.

Presenter emphasis. RAG improves access to evidence; it does not automatically establish that an answer is correct.

10. Build and govern your source data carefully.



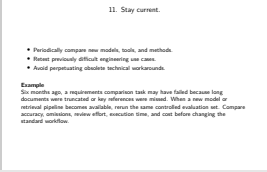
The model can be no more current or authoritative than the corpus it is given.

Explanation. Even an excellent model can produce a poor engineering answer when it receives outdated or conflicting information. Source data require ownership, quality checks, version identification, and maintenance. For retrieval systems, this also includes evaluating chunk boundaries, metadata, embeddings, and index refresh procedures. Before placing documents into any AI service, understand the service's storage and data-handling rules and the project's security requirements.

Illustrative example. Exclude superseded specifications and verify an index after each baseline update.

Presenter emphasis. Think of the knowledge base as a maintained engineering configuration item.

11. Stay current.



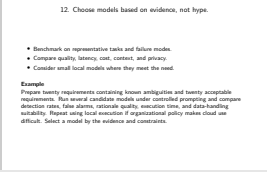
Revisit assumptions about what AI tools can and cannot do.

Explanation. Generative AI capabilities, context handling, deployment patterns, and tool integrations change quickly. A limitation identified during a previous evaluation may disappear, while a new model may create different errors. Establish a modest recurring review of available capabilities and evaluate them against your own engineering workload, not merely against vendor announcements.

Illustrative example. Reevaluate document comparison after a model or retrieval upgrade.

Presenter emphasis. Stay current experimentally, without allowing novelty to replace validation.

12. Choose models based on evidence, not hype.



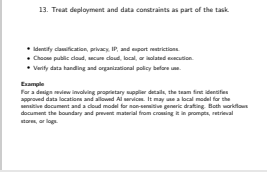
Model selection is an engineering trade study.

Explanation. There is no universally best model for every systems engineering task. A larger model may be more capable on one task while a smaller or specialized model is sufficient, faster, less expensive, or more appropriate for controlled data on another. Selection should use a representative test set, accepted reference answers, deployment constraints, and reliability measures. Ask not only how often a model succeeds, but how it fails.

Illustrative example. Benchmark cloud and local models on identical requirement-review cases.

Presenter emphasis. A published benchmark is an input to selection, not a substitute for task-specific testing.

13. Treat deployment and data constraints as part of the task.


13. Treat deployment and data constraints as part of the task.

- Identify classification, privacy, IP, and export restrictions.
- Choose public cloud, secure cloud, local, or isolated execution.
- Verify data handling and organizational policy before use.

Example

For a design review involving proprietary supplier details, the team first identifies approved data locations and allowed AI services. It may use a local model for the sensitive document and a cloud model for non-sensitive generic drafting. Both workflows document the boundary and prevent material from crossing it in prompts, retrieval stores, or logs.

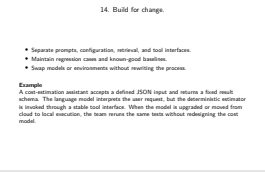
The permissible environment shapes the architecture.

Explanation. An AI workflow that works technically can still be unusable if it violates the project's data rules. Deployment decisions must be made alongside performance decisions. Public cloud, government or enterprise environments, local models, and isolated networks differ in available capability, operational controls, and data flows. Review what material may be provided, logged, retained, or transferred before beginning an experiment.

Illustrative example. Keep restricted program artifacts inside an approved execution boundary.

Presenter emphasis. Security and data governance are design requirements, not an afterthought.

14. Build for change.

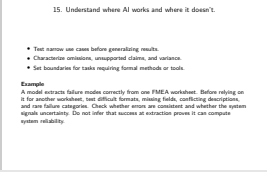


Version the prompts, data, and tests so the model can be replaced without redoing the process.
Explanation. The AI landscape will change faster than many engineered systems and organizational processes. Avoid tying critical methods to a single provider-specific conversation format or undocumented feature. Version the prompt specification and retrieval collection, use controlled interfaces for tools, and maintain repeatable test cases. Changes should be treated as modifications that require regression testing.

Illustrative example. Use one stable evaluation harness across several model providers.

Presenter emphasis. Ask the audience which part of their current workflow would break if the model were replaced tomorrow.

15. Understand where AI works and where it doesn't.



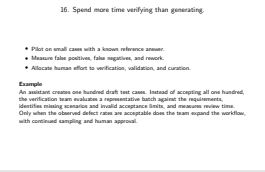
Competence is task-specific; success on one task is not evidence for another.

Explanation. Performance is uneven across tasks. A model can summarize a technical report well and still make errors in unit conversions, arithmetic, requirement attribution, or complex logical reasoning. An engineering validation claim applies to the specific task, inputs, and conditions that were tested. Study error patterns and decide what failures can be detected, contained, or escalated.

Illustrative example. Treat a successful summary as no evidence of calculation accuracy.

Presenter emphasis. Ask: In which envelope of tasks and inputs has this behavior actually been demonstrated?

16. Spend more time verifying than generating.



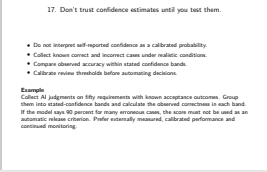
Cheap generation makes rigorous review more important.

Explanation. AI can generate large volumes of plausible engineering content rapidly, but reviewing that content often determines whether the workflow is safe and useful. Speed without quality control can create more total work. Establish acceptance tests, sample outputs, compare against reference cases, and measure the cost of correcting defects. The human role shifts toward defining the task and verifying the result.

Illustrative example. Audit a sample of AI-generated test cases before scaling production.

Presenter emphasis. Ask the audience to compare the time spent generating with the time spent reviewing and correcting on a recent task.

17. Don't trust confidence estimates until you test them.



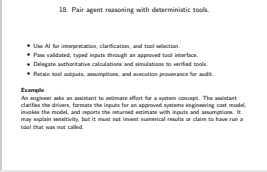
Stated confidence is not measured accuracy until it has been tested.

Explanation. A language model can sound certain while producing a false answer. A reported confidence percentage is not automatically a probability that the statement is true. Confidence can also be distorted by the model's preference to agree with the user. If a workflow relies on confidence to decide which outputs can bypass review, test that signal against verified examples and calibrate it where possible.

Illustrative example. Check whether answers labeled "90% confident" are correct about 90% of the time.

Presenter emphasis. Invite the audience to run the banded-confidence check on a small set of their own verified cases.

18. Pair agent reasoning with deterministic tools.



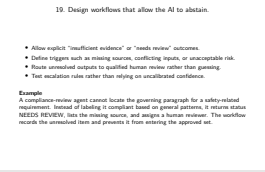
Let AI coordinate work, but let verified tools compute results.

Explanation. A generative model is valuable as a flexible interface that interprets questions and orchestrates engineering steps. But numerical calculations, simulations, database operations, traceability queries, and rule-based compliance checks should be executed by tested and governed tools. This architecture pairs language-based flexibility with reproducible computational evidence. Inputs and outputs need schemas and validation just like any other interface.

Illustrative example. An AI assistant calls an approved cost model rather than estimating cost by prose.

Presenter emphasis. Caution that an assistant's claim to have run a tool should be confirmed against the execution record.

19. Design workflows that allow the AI to abstain.



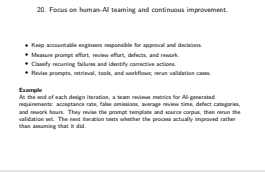
Treat "insufficient evidence" as a valid and expected output.

Explanation. An assistant that always returns a complete answer may be less useful than one that recognizes when the available evidence is insufficient. Make abstention a valid and expected outcome. Establish conditions for asking questions, identifying missing documents, referring work to a qualified engineer, or returning an explicit non-answer. In agentic workflows, that decision becomes a branch in the process rather than an apologetic sentence.

Illustrative example. Require "needs review" when a safety requirement lacks authoritative support.

Presenter emphasis. Escalation decisions should be based on tested criteria, not blindly on an uncalibrated confidence number.

20. Focus on human-AI teaming and continuous improvement.



Accountability stays with the engineer, and the process improves only as fast as it is measured.

Explanation. The long-term value is not replacing engineering judgment but making teams more effective at creating and evaluating artifacts. Keep responsibility for objectives, tradeoffs, acceptance, and final decisions with people. Track the complete process: how much effort is spent framing prompts, checking results, fixing defects, and managing disagreements. Use the measurements to improve prompts, retrieval quality, evaluation rubrics, tools, and training.

Illustrative example. Review AI defect trends after each engineering iteration and adjust the workflow.

Presenter emphasis. The destination is more defensible systems engineering decisions, not more automated output.